

Informe de Decisiones de Arquitectura

Tarea 1 – IIC3103

Nombre: Gaspar Magna Rodríguez

Índice

1. Diagrama de Arquitectura	2
2. Diagramas de Flujo	2
2.1. Flujo PRE	3
2.2. Flujo DCR	4
2.3. Flujo CIMD	5
3. Diagrama Entidad-Relación	6

1. Diagrama de Arquitectura

En el diagrama se puede apreciar una arquitectura cliente-servidor. Por un lado tenemos un cliente, que es el frontend, y por otro lado un servidor, que es el Backend, que está conectado con el Authorization Server y los 3 MCPs. Además, se tiene una base de datos desplegada en Supabase, la cual puede ser llamada por el backend ya sea para eliminar, actualizar o agregar registros.

En cuanto al stack utilizado, el frontend fue desarrollado en Vite + React, y el backend en FastAPI. Por otra parte, el despliegue del frontend se hizo en Vercel, mientras que el del backend se hizo en Render. Se eligieron estos servicios de despliegue debido a la simplicidad de estos y porque la página no tiene una gran cantidad de usuarios y acciones por minuto, por lo que los planes Free Tier bastan. Cabe mencionar que el backend, al estar desplegado en Render, tiene el tradeoff de tener cierto cold start cuando no se usa en cierto rango temporal.

Una decisión arquitectónica importante fue la de implementar un proxy para que el backend pudiese ser llamado desde la URL del frontend con el endpoint `/api`. Esto fue necesario debido a que algunos navegadores no permiten guardar la cookie de sesión cuando esta proviene de un dominio distinto al de la página. Justamente ese fue el caso, pues se utilizaron servidores distintos para desplegar frontend y backend, por lo que fue necesaria esta acción.

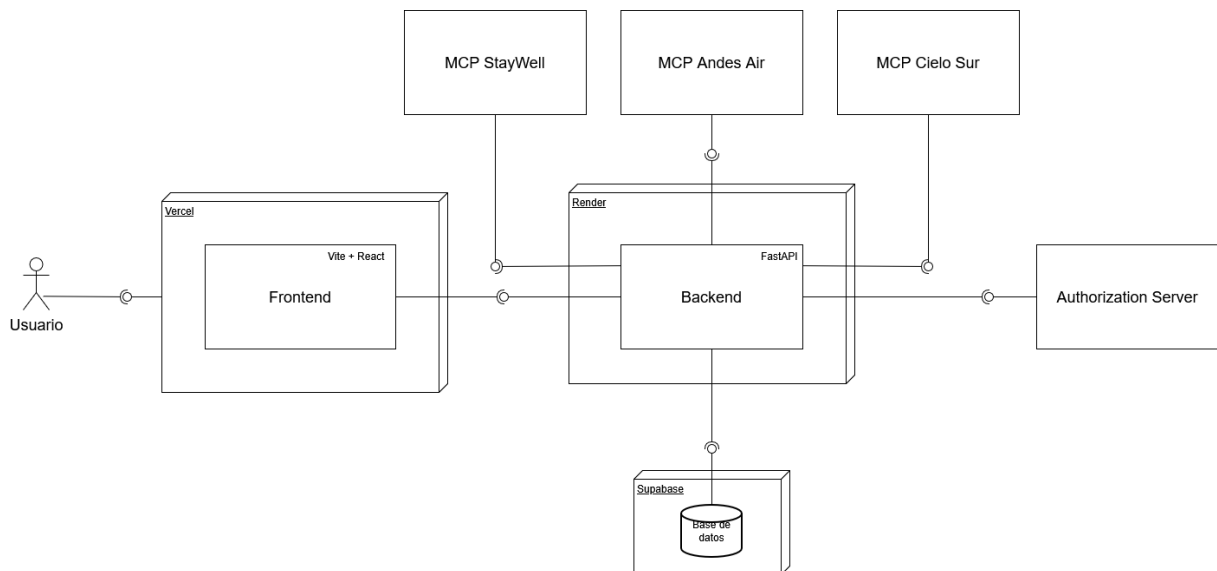


Figura 1: Diagrama de arquitectura del sistema.

2. Diagramas de Flujo

Los tres flujos comparten muchos elementos. Todos inician con la selección del botón de conectar del MCP respectivo. Esta acción gatilla un GET a la ruta `/mcp/{server_name}/connect` del backend que es necesaria para empezar el flujo de conexión. Este proceso comienza con la obtención de metadata importante para la conexión, la cual contiene elementos como el `authorization_endpoint`, `token_endpoint`, entre otros.

Luego viene la fase que es distinta en cada MCP, que es el proceso de generación u obtención y posterior guardado de los `client_id` y `client_secret_enc`. Las diferencias en cada proceso se explicarán en las secciones de cada tipo de Auth.

A continuación se generan el `state` y `code_verifier` para ser guardados en la base de datos. Con estos valores, se construye la URL de autorización que redirige al AS, lugar en el cual se debe

aceptar un consentimiento para acceder al MCP de turno.

Después de este proceso, se desarrolla un flujo necesario para seguir el protocolo PKCE. En este se elimina el state de la base de datos y, por su parte el AS, verifica el `code_verifier` corresponda a el `code_challenge` enviado anteriormente. Al comprobar que coinciden, se obtiene un `access token` necesario para poder llamar las tools.

Finalmente, se guardan los tokens en la tabla correspondiente y, al confirmar el guardado, se redirige al frontend donde se visualiza que la conexión con el MCP se ha concretado.

2.1. Flujo PRE

En este tipo de autenticación, el `client_id` y el `client_secret` se generan mediante un registro previo realizado en el AS proporcionado. Estos valores deben guardarse manualmente en la base de datos, en particular, en la tabla `mcp_servers` en la fila correspondiente al servidor de Andes Air:

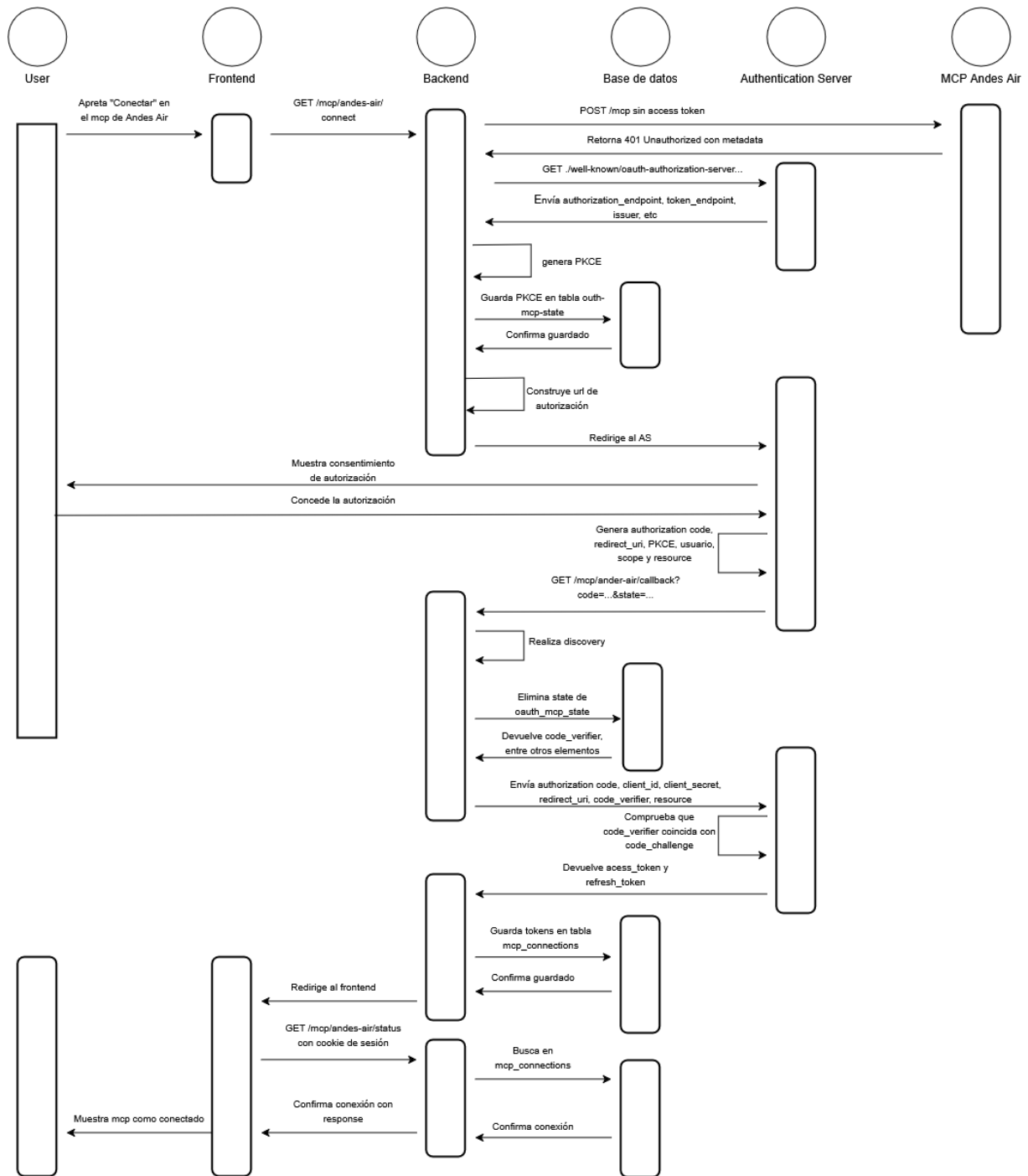


Figura 2: Flujo de conexión mediante Pre-Registered Client (PRE).

2.2. Flujo DCR

En este tipo de autenticación, la forma de obtener los `client_id` y `client_secret` es distinta de la de PRE. En este caso, se debe registrar el client haciendo un POST a `registration_endpoint` (este valor es exclusivo de DCR) obtenido a partir de la metadata. Una vez realizada esta request, en la response obtenida de esta se pueden conocer los valores necesarios para poder conectarse al servidor StayWell.

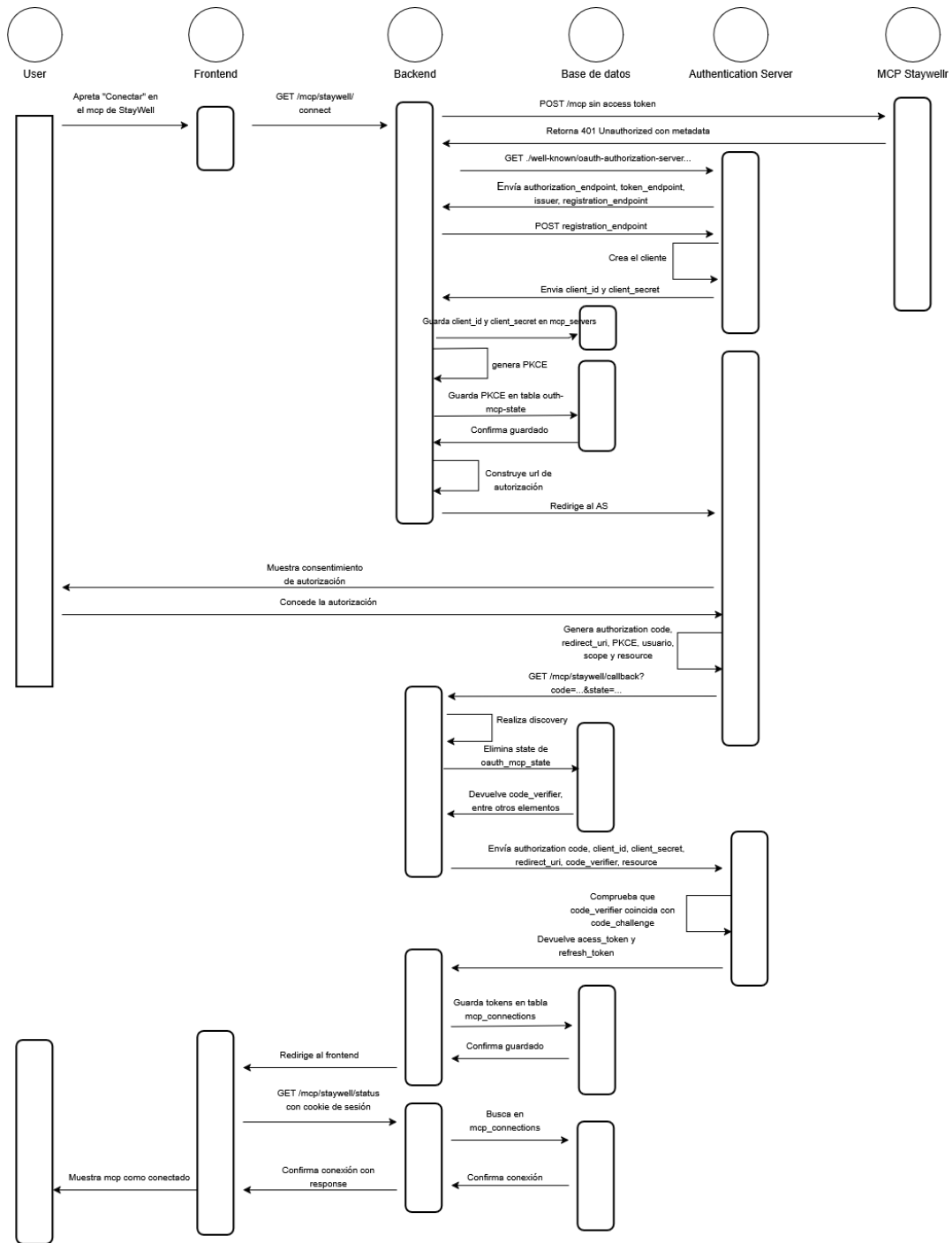


Figura 3: Flujo de conexión mediante Dynamic Client Registration (DCR).

2.3. Flujo CIMD

En este tipo de autenticación el `client_id` es un endpoint. En particular: <https://integratrip-temp.vercel.app/api/.well-known/oauth-client-metadata.json>. En CIMD no es necesario un valor de `client_secret`, por lo que este valor se mantiene nulo en la base de datos. Luego de guardar el `client_id` en la tabla `mcp_servers`, continúa el flujo común.

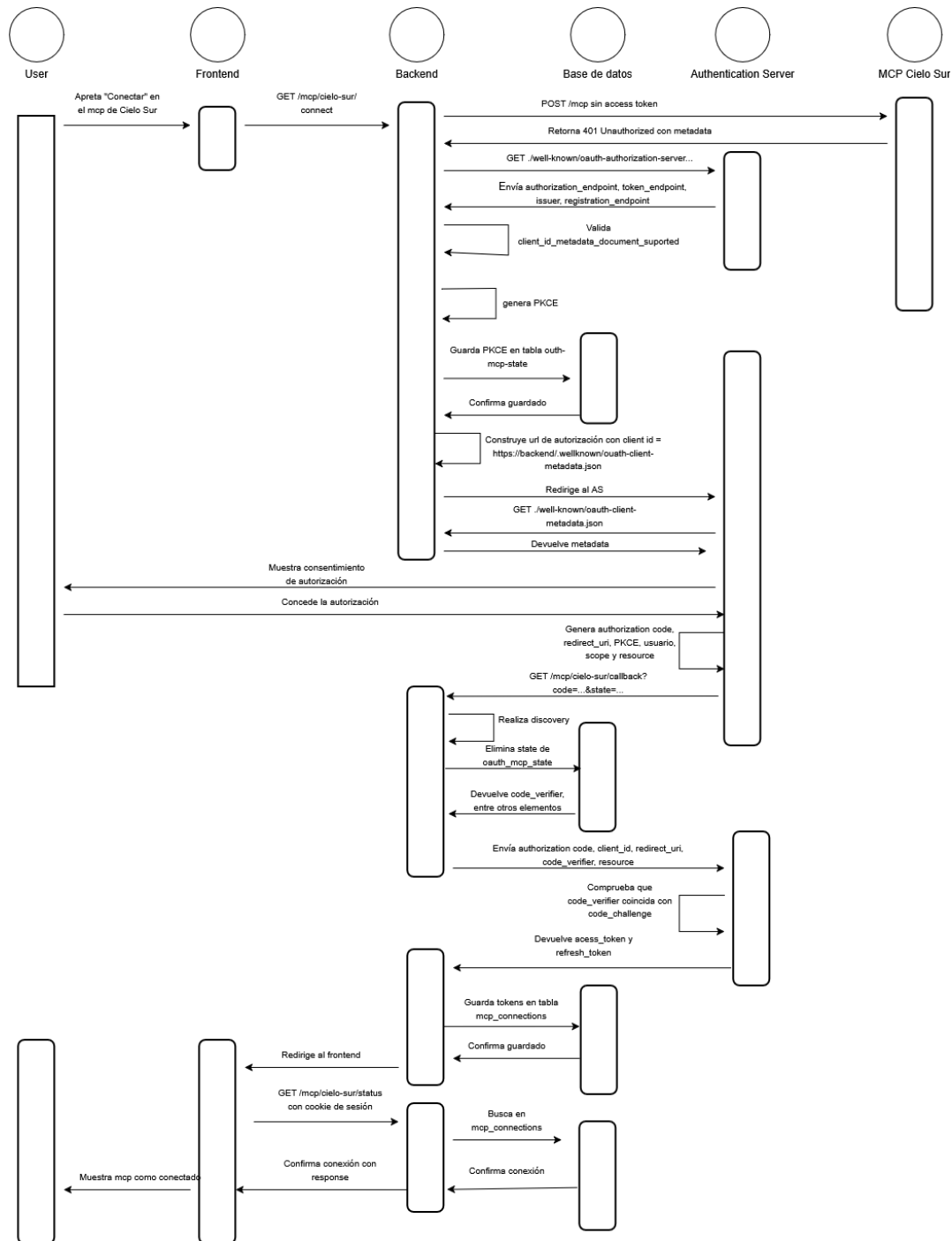


Figura 4: Flujo de conexión mediante Client ID Metadata Document (CIMD).

3. Diagrama Entidad-Relación

Se tienen 5 tablas: `oauth_login_state`, `mcp_connections`, `users`, `oauth_mcp_state` y `mcp_servers`.

En la tabla `oauth_login_state` se guarda toda la información del estado del login. En particular, elementos relevantes para seguir el protocolo PKCE como lo son el `state` y `code_verifier`. Esta tabla no contiene ninguna llave foránea, ya que su único objetivo es guardar la información necesaria para el inicio de sesión.

En la tabla `mcp_connections` se guarda la información de cada conexión. Entre estos elementos

se encuentran el id del usuario que se conectó, el id del servidor MCP, los tokens que se obtienen luego de haber concretado la conexión, entre otros elementos.

En la tabla users simplemente se guarda el as_subject y el email de las personas que iniciaron sesión en el Authorization Server. Un usuario puede estar presente en varias conexiones con servidores MCP y varios estados de autenticación relacionados con estos.

En la tabla mcp_servers se guarda la información de cada servidor. En esta tabla se guardan el client_id y el client_secret_enc, aunque a veces algunos de estos valores pueden ser nulos dependiendo del servidor y su tipo de autorización. Un servidor MCP puede estar presente en varias filas de mcp_connections y de oauth_mcp_state, ya que pueden existir varias conexiones a un mismo servidor.

Finalmente, se tiene la tabla oauth_mcp_state. Esta contiene los elementos que se obtienen a partir de un intento de autenticación con un MCP. Entre ellos, el id del usuario que se quiere autenticar, el id del servidor, y el state y el code_verifier, que son primordiales para seguir el protocolo PKCE.

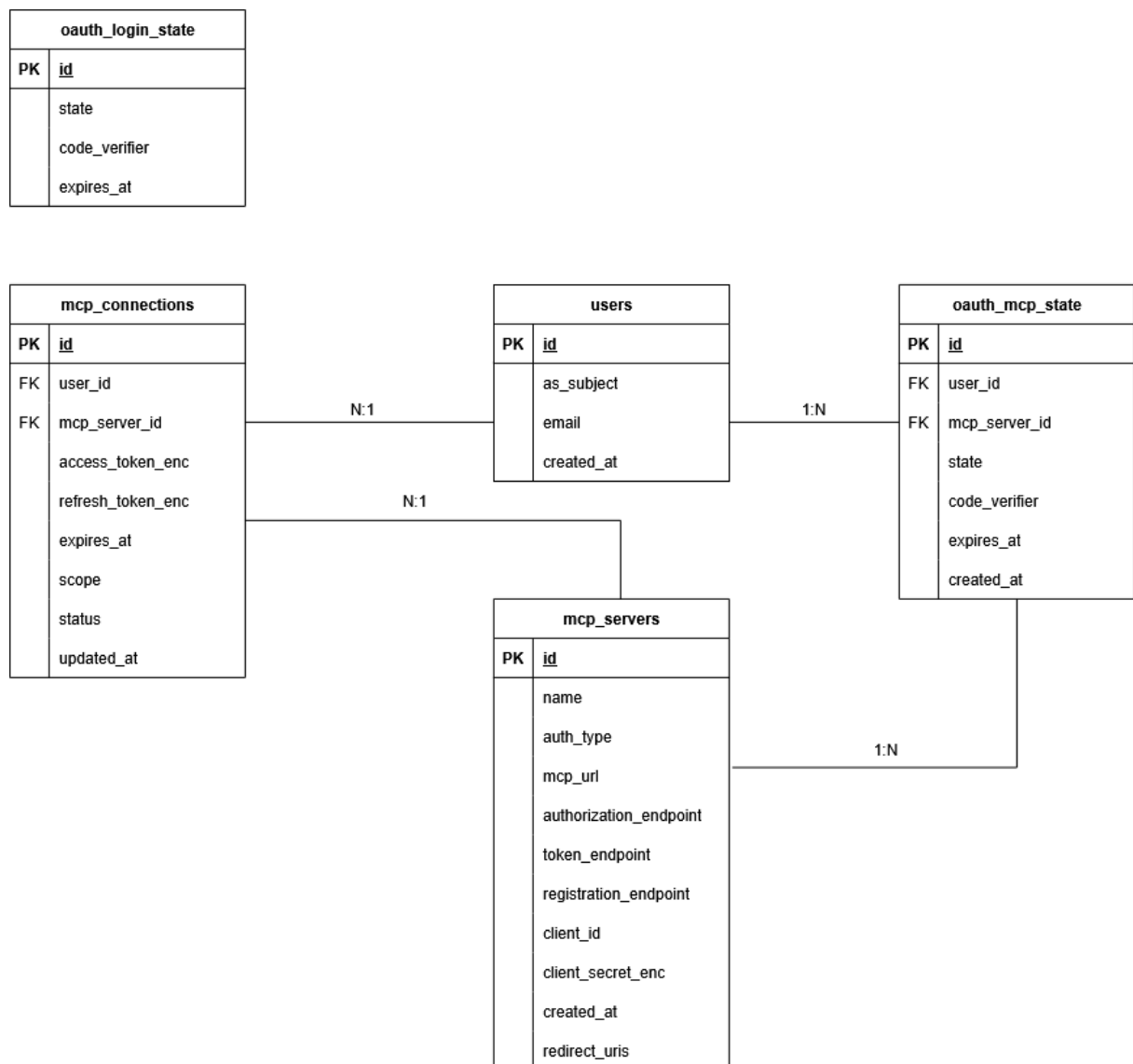


Figura 5: Diagrama entidad-relación del sistema.